

Learning Spine-Assisted Locomotion: A reinforcement learning comparison of spined and rigid quadruped robots

Rahul Rajesh

Sun Robotics Lab · FURI · Arizona State University

Faculty mentor: Jiefeng Sun

Roadmap

- Biomechanical motivation for a flexible spine
- How the spine is approximated in simulation
- MDP design: actions, observations, rewards, terminations
- PPO training setup, metrics, and current progress
- Planned rigid-vs-spined quantitative comparison



Core claim

Main variables: reward convergence, episode stability, forward speed, and energy

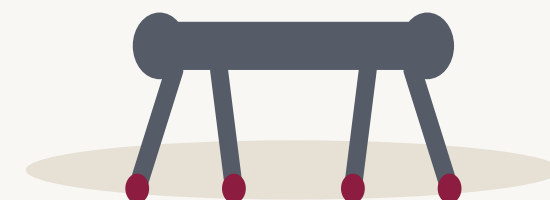
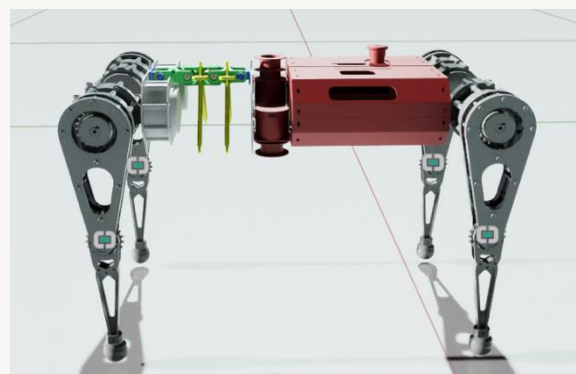
Question: How does a flexible spine affect reinforcement learning performance and locomotion efficiency in a simulated quadruped robot?

Why add a spine at all?

Motivations

Biological intuition

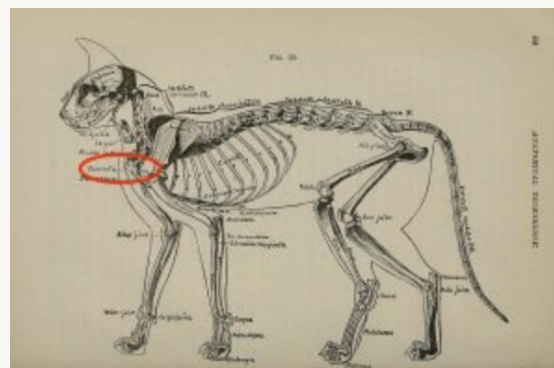
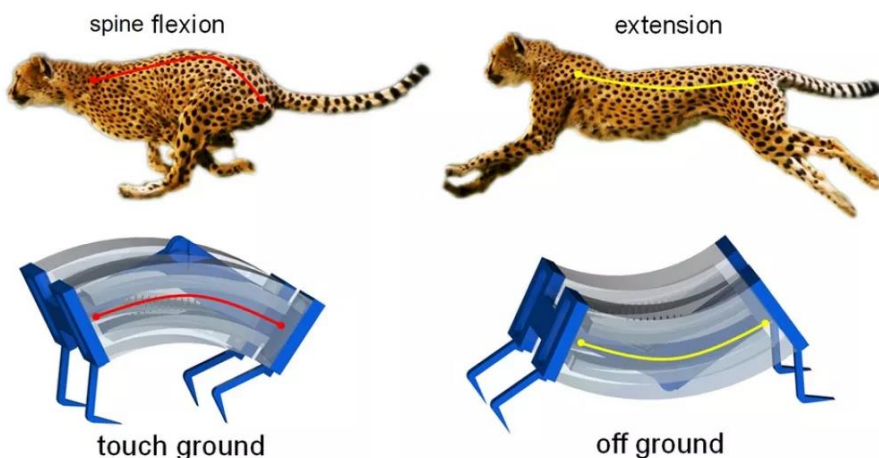
- Spine flexion increases stride length
- Redistributes loads across the body
- Adds extra degrees of freedom
- Potential gains in stability, agility, efficiency



Rigid spine limits body coordination



Flexible spine adds controllable body shape



What the study tests

Controlled comparison

- Same task, terrain, reward, PPO
- Only variable: morphology (rigid vs spined)

My contributions this semester

From isolated spine debugging to full-quadruped integration

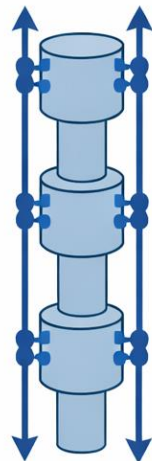
Contribution

Engineering path

- Helped model and validate spined quadruped in Isaac Sim
- Found tendon simulation unstable → replaced with equal and opposite force-based model
- Verified model on isolated spine environment
- Integrated spine into full quadruped RL pipeline

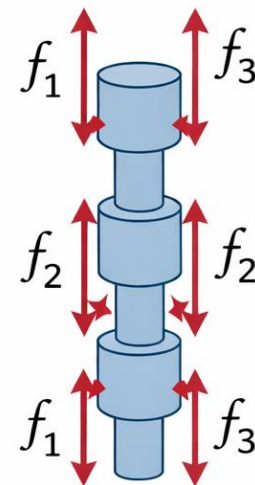
- Real system: 3 tendons along the spine
- Simulation: map tendon tensions → net force + torque
- Goal: preserve mechanical effect, not exact cable physics

Why the actuation model matters



Real system:
Tendon dynamics

Convert tensions →
Net force + torque



Approximation:
Net force + torque

Research question and hypotheses

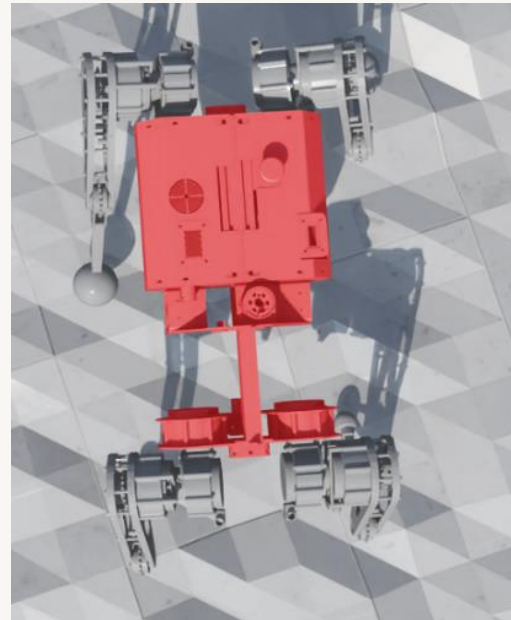
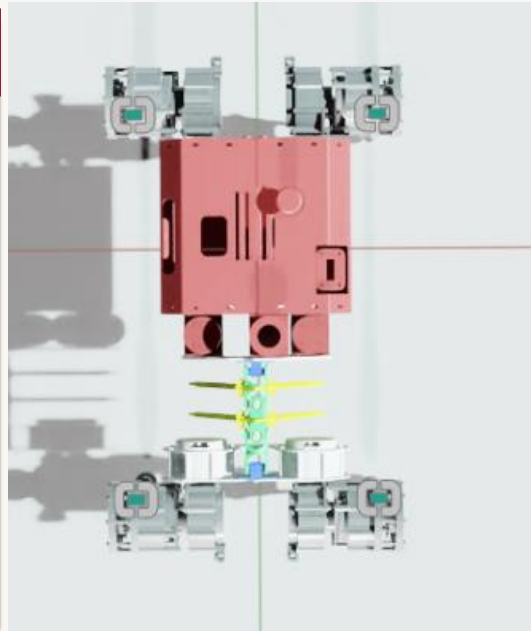
Study design

Research question

How does a flexible spine affect reinforcement learning performance and locomotion efficiency in a simulated quadruped robot?

Primary hypothesis

- H1: Higher final reward
- H2: Faster convergence measured by reward and episode-length growth
- H3: higher speed, lower torque norm, lower energy per distance.



Controlled variables

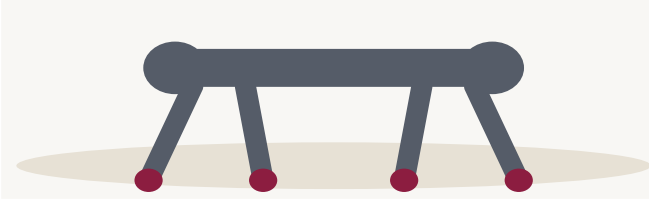
- Environment: terrain, commands
- Training: PPO, reward, duration
- Robustness: domain randomization

Comparing rigid and flexible spines under matched training conditions to ensure no advantages or skew.

Robot system overview

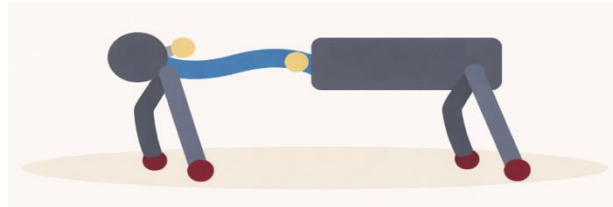
Robot

Rigid-body baseline

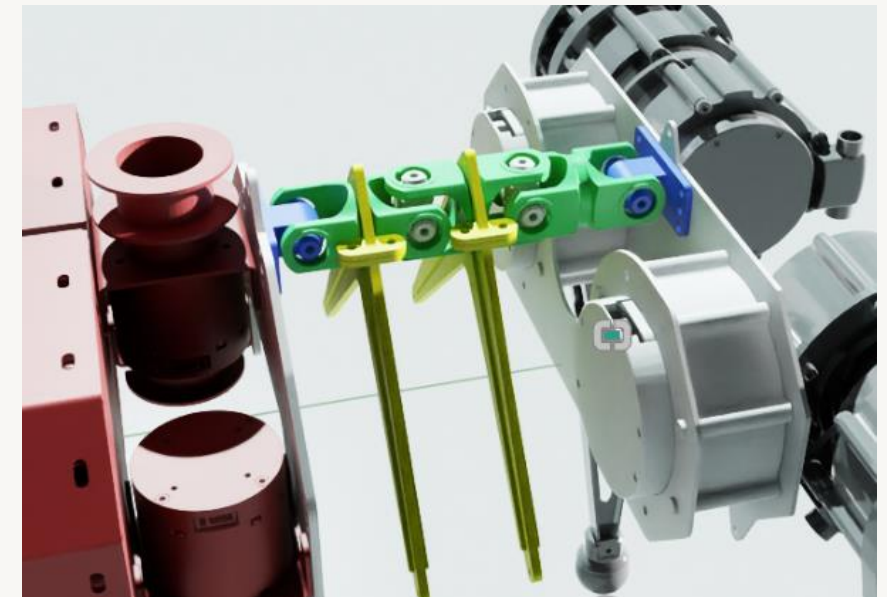


- Four legs, 3 actuated joints per leg.
- Torso treated as a rigid box with no controllable spine deformation.
- Serves as the morphology-matched baseline for RL comparison.

Spined quadruped



- Same leg topology plus an articulated spine segment chain
- Spine is actuated through three tendon-equivalent force inputs
- Goal: test whether extra spinal control changes learned locomotion



RL formulation as a Markov Decision Process

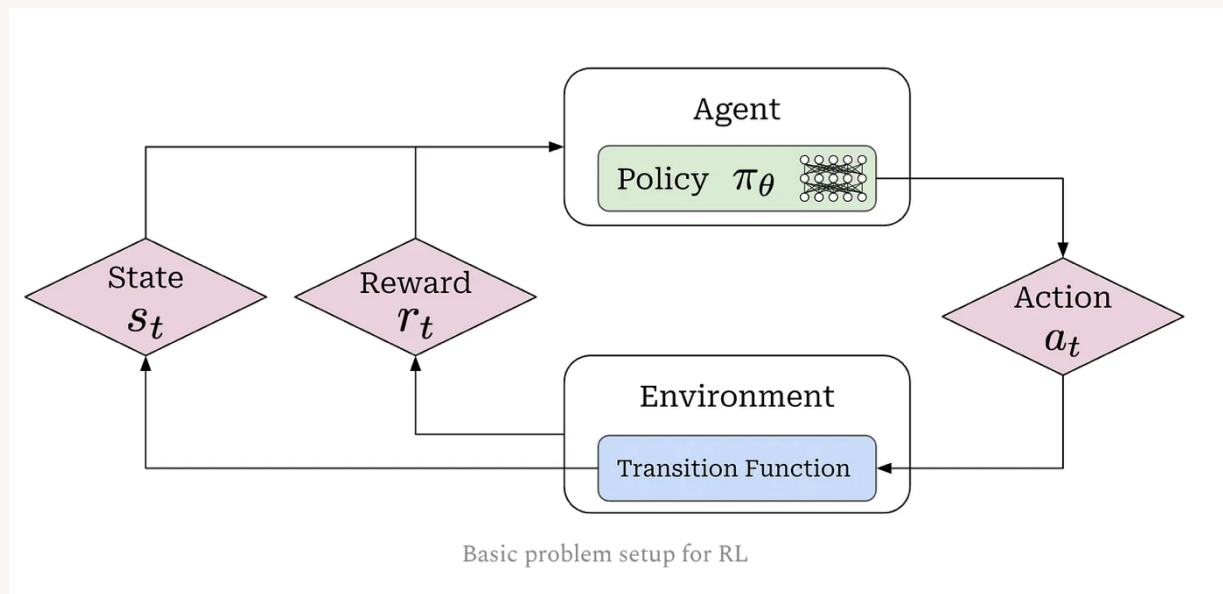
MDP

MDP definition

- **Observation:** joint states, base motion, gravity, EE pose/velocity, previous action
- **Action:** 12 leg targets + 3 spine forces
- **Reward:** tracking + gait – penalties
- **Termination:** timeout, illegal contact, out-of-bounds

Control loop used in training

- Policy outputs **15D action vector (12 joints + 3 spine forces)**.
- Actions applied through physics simulation
- Rewards computed from locomotion objectives
- PPO updates policy from rollout batches



$$a_t \sim \pi_\theta(a_t \mid o_t)$$

$$(o_{t+1}, r_t) \sim \mathcal{P}(o_{t+1}, r_t \mid o_t, a_t)$$

$$\theta \leftarrow \text{PPO}(\theta, \{o_t, a_t, r_t\})$$

Action space and actuator model

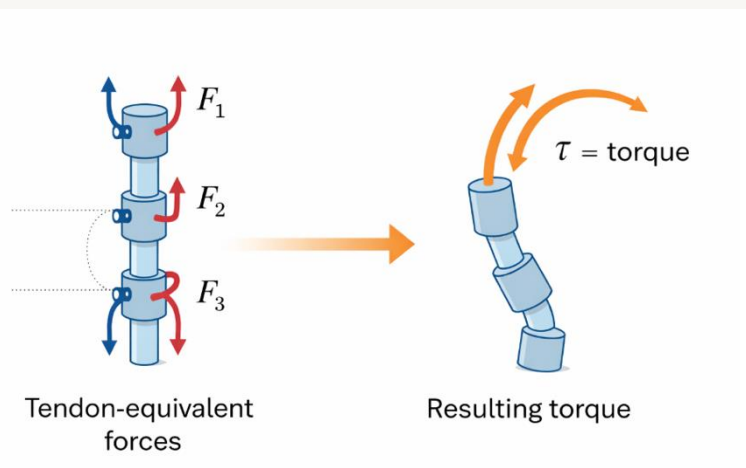
Actions

Control variables

- Leg control: 12 commands (3 joints $\times$ 4 legs).
- Spine control: 3 tendon-equivalent force inputs.
- Total action vector (12 joint targets + 3 spine forces):

$$a_t \in \mathbb{R}^{15}$$

$$f_i = \text{clip}(\alpha_i a_i + \beta_i, 0, f_{\max})$$



$$\tau_{\text{net}} = \sum_{i=1}^3 r_i \times F_i$$

Actuation mapping

- Leg motors use PD-controlled joint position targets defined in Isaac Lab actuator configuration
- Spine inputs are clipped and transformed into nonnegative cable-like tensions to approximate tendon actuation
- These tensions are then converted into a net force and net torque before being applied to the spine bodies.

Observation space and what the policy can sense

Observations

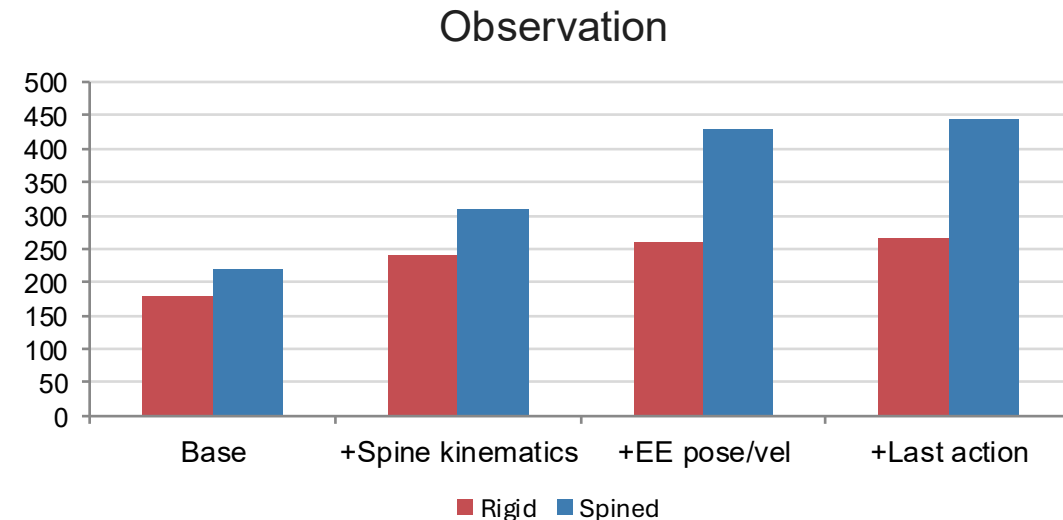
State vector structure

$$o_t = [q, \dot{q}, v_b, \omega_b, g_b, a_{t-1}, x_{ee}, \dot{x}_{ee}]$$

- q includes **all joints (legs + spine)**
- Base linear velocity $\mathbf{v}_b$ and angular velocity $\mathbf{w}_b$ capture gross body motion.
- Projected gravity ($\mathbf{g}_b$) provides orientation relative to the world frame.
- Previous action ($\mathbf{a}_{(t-1)}$) helps the policy model actuator dynamics and improves temporal consistency
- End-effector pose/velocity improve observability of body shape

Observation design question

- Too few observations $\rightarrow$ partial observability
- Too many observations $\rightarrow$ higher-dimensional inputs increase learning complexity



Reward, termination, and quantitative metrics

Objectives

Reward template

$$r_t = w_v r_{\text{vel}} + w_g r_{\text{gait}} + w_a r_{\text{air}} - \lambda_\tau \|\tau\| - \lambda_E E_t - \lambda_c C_t$$

- r_{vel} : velocity tracking
- r_{gait} : symmetry and stability
- r_{air} : stepping behavior, air time
- $\|\tau\|$: effort penalty
- E_t : Energy proxy
- C_t : Contact penalty
- For the spined robot, bad cable-force behavior and excessive body-angle deviation can be penalized explicitly.

Termination and measurements

- Energy = power over time
- Terminate on failure conditions.
- Without proper contact penalties, policies may exploit non-physical strategies (dragging or collapsing forward)
- Reported metrics: reward, episode length, speed, efficiency, and effort.

Energy :

$$E_t \approx \sum_j |\tau_j \dot{q}_j| \Delta t$$

Torque Norm
(effort metric) :

$$\|\tau\|_2 = \sqrt{\sum_j \tau_j^2}$$

Tendon approximation with equal-and-opposite forces

Mechanics

Why this approximation is needed

- Spatial tendons cannot be defined in articulation
- The workaround → approximating net wrench (force + torque)

Wrench construction

- Apply (F_{net} , T_{net}) to Body A and ($-F_{\text{net}}$, $-T_{\text{net}}$) to Body B
- Off-axis tendon positions r_i generate a bending moment even when the net external force is 0, which preserves internal-force consistency
- This gives the policy a controllable spine effect without unstable tendon simulation.

$$F_i = f_i \hat{d}_i$$

→ $\hat{d}_i$ is tendon direction vector

$$F_{\text{net}} = \sum_{i=1}^3 F_i$$

$$\tau_{\text{net}} = \sum_{i=1}^3 r_i \times F_i$$

Why PPO is the right learning algorithm here

Optimization

Actor-critic structure

- **Actor:** outputs 15D continuous actions (legs + spine)
- **Critic:** estimates value to stabilize learning
- **Why:** reduces variance in noisy environments

$$\pi_{\theta}(a_t | o_t), \quad V_{\phi}(o_t)$$

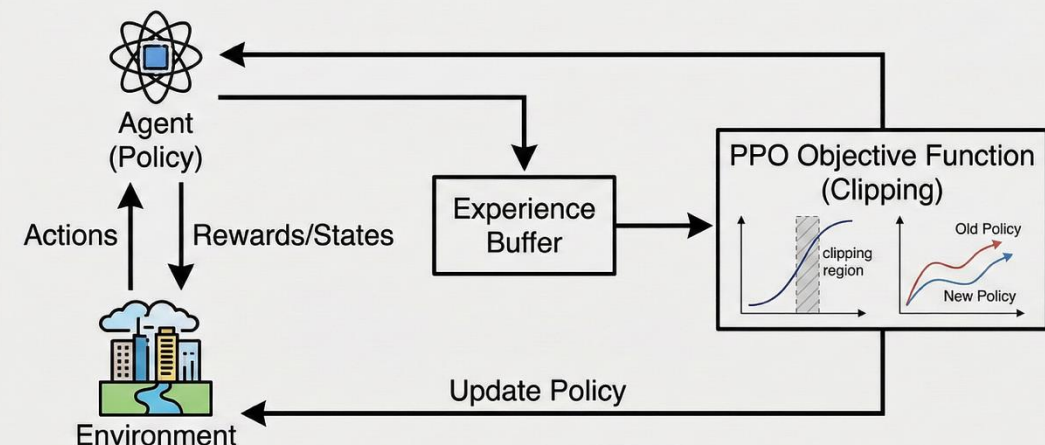
PPO objective

- Uses **clipped updates** to avoid large policy changes
- Stable for **contact-rich dynamics**
- Works well for **high-dimensional continuous control**

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | o_t)}{\pi_{\theta_{\text{old}}}(a_t | o_t)}$$

Proximal Policy Optimization (PPO) Algorithm



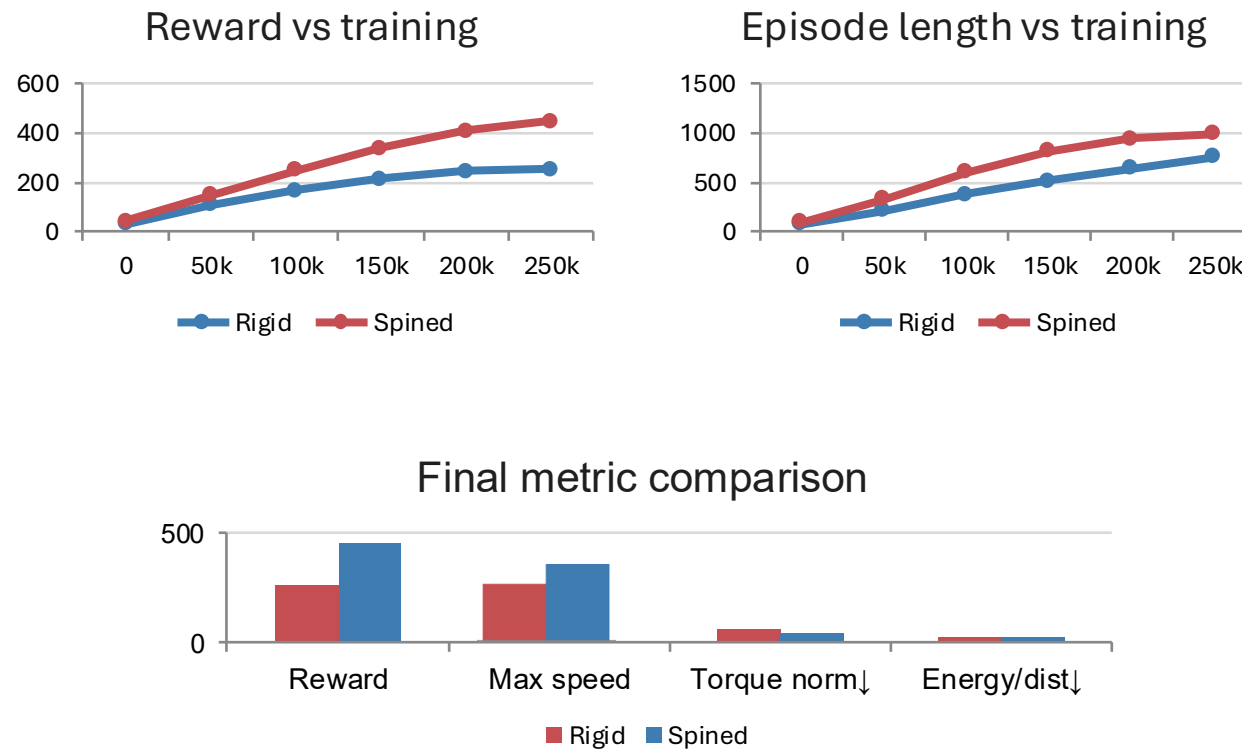
Experimental protocol for a fair morphology comparison

Protocol

Controlled setup

- Same terrain generation and same command ranges.
- Same reward structure and termination logic.
- Same PPO algorithm, rollout length, and training duration.
- Same reset distributions and domain randomization settings.

Planned quantitative outputs



Final results will be updated with completed training runs

Current progress and what the debugging revealed

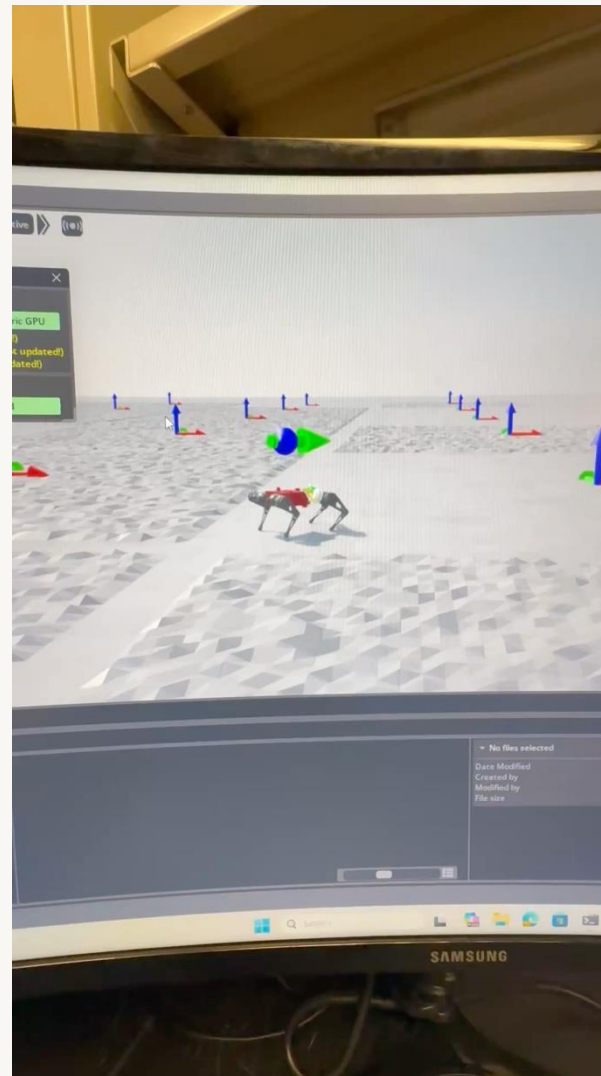
Progress

What is already working

- Training and simulation rollouts now run for the spined quadruped.
- Custom spine action term successfully applies tendon-equivalent external wrenches.
- PPO training has already been validated on the isolated spine environment.

Some debug findings

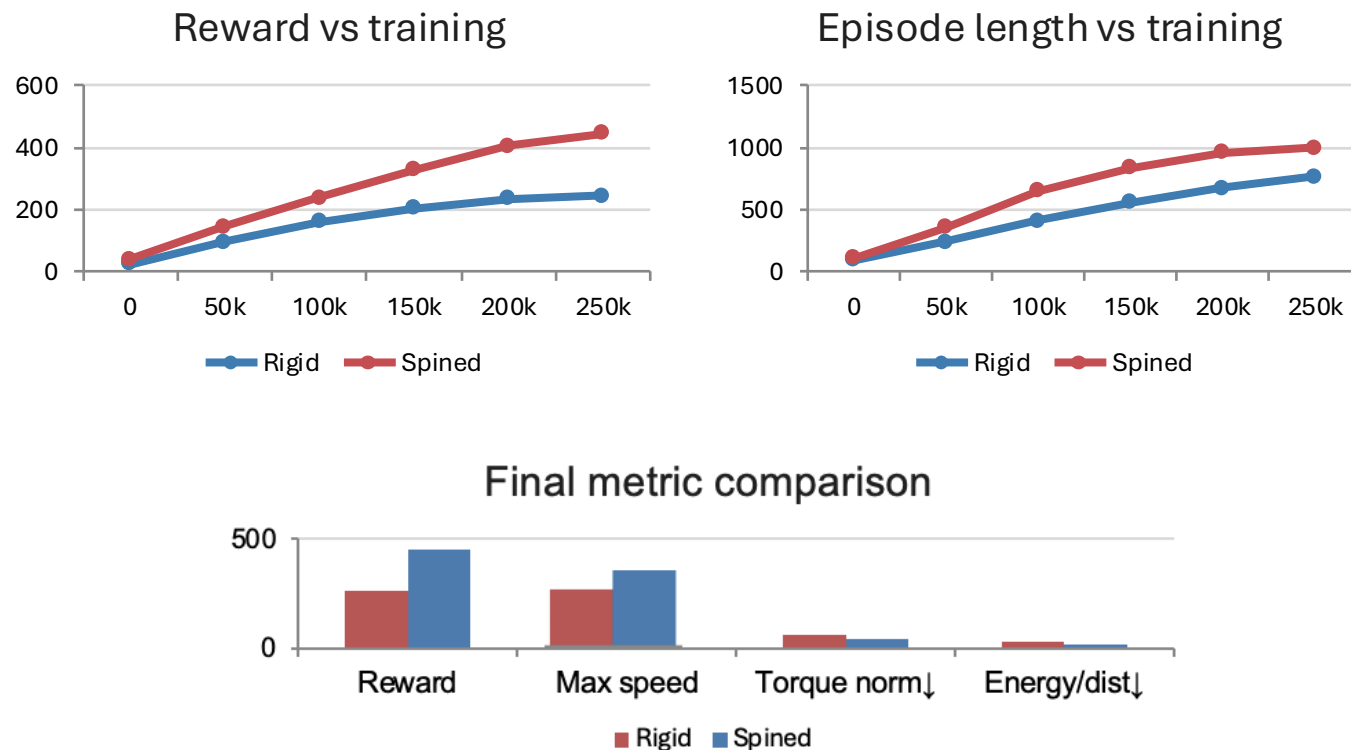
- The robot would fall forward during zero_agent and random agent → spine joints weren't active
- Walked with a left side arch after training → center of mass on pelvis was misplaced
- Spatial Tendons → Equal and Opposite Forces
- Current Issues:
 - Not turning in designated direction
 - Not going as fast as it should be



Quantitative story for the final comparison

Results

Main results figures



Interpretation: if the spine really adds useful controllability, the spined robot should learn faster, survive longer, move faster, and do so with lower effort.

How to present this honestly

- These shapes are hypotheses / placeholders until final rigid-vs-spined training runs finish.
- What final runs need to prove: whether spinal flexibility measurably improves reward and efficiency.

Takeaways, next runs, and what success looks like

Close

Takeaways

- This project is not just about training “a robot policy”; it is about testing whether more spinal degrees of freedom changes what RL can learn.
- The spine is modeled in a simulation-stable way through equal-and-opposite wrenches that approximate tendon action.
- The final deliverable is a controlled rigid-vs-spined comparison using matched RL conditions and quantitative metrics.

Immediate next steps

- Complete stable training for the spined quadruped.
- Train the rigid baseline under the exact same setup.
- Generate final reward, episode-length, torque, speed, and energy plots.
- If the spine wins under controlled comparison, the next phase is more complex terrain and eventual sim-to-real transfer.